Qwen

Архитектура Трансформера: Руководство по внутреннему устройству современных языковых моделей

В данном исследовании представлен всесторонний анализ принципов работы искусственного интеллекта, основанного на архитектуре Transformer. Целью является предоставление глубокого, но доступного концептуального понимания механизма работы таких моделей, охватывающего два ключевых цикла: обучение на больших массивах данных и последующая генерация ответов на пользовательские запросы. Исследование структурировано таким образом, чтобы последовательно разобрать каждый этап, используя минимальное количество математических формул и максимальное количество функциональных аналогий и визуальных концепций. Особое внимание уделяется роли параметров, токенизации, векторного представления данных, механизма внимания и специфической структуре слоев, которая лежит в основе мощи современных языковых моделей [2] [4]. Данный отчет предназначен для аудитории со средним техническим уровнем знаний и стремится не просто перечислить компоненты, а показать, как они взаимодействуют для достижения сложных задач в области обработки естественного языка [16].

Процесс обучения: Как модель "учится" на огромных объемах текста

Процесс обучения модели на архитектуре Transformer представляет собой фундаментальный этап, на котором модель приобретает способность понимать и генерировать человеческий язык. Этот процесс кардинально отличается от традиционного программирования, где действия машины строго задаются человеком. Вместо этого, обучение — это процесс "развития", при котором модель самостоятельно выявляет статистические закономерности и семантические связи в огромных массивах текстовых данных [25]. Чтобы понять этот процесс, необходимо разобраться в трех ключевых элементах: датасете,

параметрах и механизме оптимизации, который приводит к формированию этих параметров.

Первым шагом является подготовка и использование обучающего набора данных [12]. В отличие от задач, где требуется помеченная выборка, в обучении больших языковых моделей (LLM) основным источником знаний является сырой текстовый корпус, состоящий из книг, веб-страниц, новостей и другой публичной информации [27]. Эти данные обычно делятся на две части: тренировочная (обучающая) выборка и тестовая выборка, например, в соотношении 80% на 20% [12]. Тренировочный набор данных используется для фактического обучения модели, то есть для корректировки ее внутренних параметров. Тестовый набор данных, напротив, никогда не видит модель во время обучения; он служит для объективной оценки того, насколько хорошо модель освоила материал и готова к работе с новыми, ранее не встречавшимися примерами. Этот подход позволяет избежать переобучения, когда модель начинает механически запоминать обучающие примеры, а не учиться общим принципам [33].

Следующим важным шагом является токенизация — процесс преобразования непрерывного потока текста в последовательность дискретных единиц, называемых токенами [11]. Это можно сравнить с тем, как мы читаем книгу: слова, знаки препинания и даже части слов рассматриваются как отдельные символы или смысловые единицы. Для каждой такой единицы создается уникальный числовой идентификатор. Например, предложение "The animal didn't cross the street" будет разбито на токены: ["The", "animal", "didn't", "cross", "the", "street"]. Затем эти токены могут быть преобразованы в их числовые эквиваленты, которые модель может обрабатывать. Этот числовой код затем преобразуется в высокоразмерный вектор, называемый "эмбеддингом" [1]. Эмбеддинг — это не просто номер токена; это сложный числовой код, который кодирует в себе множество аспектов значения самого слова, его синонимов, антонимов и контекста, в котором оно часто встречается. Таким образом, семантически близкие слова будут иметь похожие векторы-эмбеддинги в многомерном пространстве [34]. Именно эта концепция позволяет модели работать не с абстрактными символами, а с их математическими представлениями, что открывает возможности для сложных манипуляций.

Центральным понятием в обучении является сама модель и ее параметры. Параметры — это огромный массив чисел, также известный как веса, которые модель "настраивает" в процессе обучения [3]. Можно провести аналогию с

настройками профессиональной программы для обработки фотографий. Сама программа (архитектура модели) определяет, какие инструменты (математические операции) доступны. А параметры — это конкретные значения этих настроек (например, яркость, контраст, уровень красного цвета). Одни и те же инструменты, но с разными настройками, дают совершенно разные результаты. Аналогично, один и тот же архитектурный шаблон трансформера, но с разными наборами параметров, превращается в совершенно разные модели (например, одна может стать переводчиком, а другая — художником-писателем). Эти параметры не хранят знания напрямую, как страницы в книге. Они определяют, как математические операции должны быть выполнены для преобразования одного вектора-эмбеддинга в другой, более информативный [6]. Например, они указывают, как нужно смешивать информацию из разных частей предложения, чтобы понять, к чему относится местоимение "оно" [1].

Процесс обучения, или оптимизации, — это итеративный алгоритмический процесс, направленный на поиск "правильных" значений для этих параметров. Он состоит из нескольких повторяющихся шагов: 1. Прямое распространение **(Forward Pass):** На каждом шаге модель получает на вход часть обучающего набора данных (например, целое предложение) и пытается сделать прогноз. Она проходит через все слои своей архитектуры и генерирует свой собственный выход, например, следующее слово в предложении или переведенный текст [13]. 2. Вычисление ошибки **(Loss Calculation):** Полученный моделью результат сравнивается с правильным, ожидаемым ответом из обучающего набора данных. Разница между этими двумя результатами измеряется с помощью специальной функции, называемой функцией потерь [3]. Если модель сказала "кошка", а правильный ответ был "собака", функция потерь вернет большое число, сигнализируя о большой ошибке. Если же она сказала "кот", ошибка будет меньше. 3. Обратное распространение ошибки**:** Это сердце процесса обучения. Ошибка, рассчитанная на предыдущем шаге, "перетаскивается" обратно через всю архитектуру модели [3] [13]. Алгоритм вычисляет, насколько каждый отдельный параметр (вес) вносил вклад в эту общую ошибку. Это делается с помощью вычисления градиентов — математического инструмента, который показывает, в каком направлении и с какой скоростью изменить каждый параметр, чтобы уменьшить ошибку. 4. Оптимизация **(**Обновление параметров**):** После того как влияние каждого параметра на ошибку установлено, модель корректирует их значения. Параметры, которые способствовали большой ошибке, немного уменьшаются, а те, что помогали делать хороший прогноз, немного увеличиваются. Этот процесс выполняется с помощью алгоритма оптимизации, такого как Adam или

SGD. Шаг изменения очень маленький, чтобы не "перескочить" точку минимума ошибки.

Эти четыре шага повторяются миллионы или даже миллиарды раз на всех примерах из обучающего набора данных. Со временем, по мере накопления опыта, модель подбирает такие наборы параметров, которые позволяют ей минимизировать среднюю ошибку на всем корпусе текста. По сути, она выучивает статистические распределения языка: какие слова чаще всего встречаются вместе, как строятся предложения и как связаны между собой разные понятия. Этот процесс можно сравнить с изучением иностранного языка в детстве: ребенок не изучает правила грамматики в учебнике, а просто погружается в среду, где язык используется постоянно, и со временем начинает говорить правильно, потому что так было на практике. Важно отметить, что обучение — это крайне ресурсоемкий процесс, требующий огромных вычислительных мощностей и времени, поскольку каждая итерация требует выполнения сотен миллионов математических операций [15]. Существуют и более продвинутые методы обучения, такие как прогрессивное удаление слоев (Progressive Layer Dropping), которые направлены на ускорение этого процесса, но базовый принцип остается неизменным [15]. Таким образом, после длительного периода обучения модель становится "выполненной" и готовой к решению конкретных задач, будь то перевод текста, ответ на вопрос или написание эссе.

Генерация ответа: Путь запроса через архитектуру Трансформера

Генерация ответа — это второй фундаментальный цикл работы модели на архитектуре Transformer, который происходит после завершения ее обучения. Если обучение — это процесс "приобретения знаний", то генерация — это процесс "применения этих знаний для решения конкретной задачи". Когда пользователь вводит запрос, модель должна не просто найти точный ответ в своей памяти (что невозможно из-за огромного размера параметров), а сгенерировать новый, осмысленный текст, который наиболее вероятно соответствует смыслу запроса. Этот процесс представляет собой сложную, но детерминированную последовательность вычислений, которую можно

проследить от первого символа запроса до последнего сгенерированного слова.

Весь процесс начинается с токенизации запроса. Входная строка текста от пользователя немедленно разбивается на последовательность токенов, точно так же, как это происходило на этапе обучения [11]. Например, запрос "Какова столица Франции?" будет преобразован в токены: ["Какова", "стол", "##и", "##ца", "Фран", "##ции", "?"]. Каждый токен затем преобразуется в свой уникальный числовой идентификатор. Эти идентификаторы являются входными данными для модели.

Следующий шаг — преобразование в векторы (эмбеддинги). Каждый числовой идентификатор токена используется для поиска соответствующего ему вектора-эмбеддинга в специальной таблице, которая является частью обученных параметров модели. Таким образом, вся последовательность токенов запроса преобразуется в последовательность высокоразмерных векторов [1]. Эти векторы — это не просто коды, а сложные математические объекты, содержащие в себе семантическую информацию о каждом слове. Важнейшей особенностью, которая сохраняется на этом этапе, является порядок слов. Поскольку трансформеры обрабатывают все токены параллельно, а не последовательно, им необходим дополнительный способ понимать порядок. Эта информация добавляется на этапе создания эмбеддингов через процедуру, называемую позиционным кодированием [3]. Каждому эмбеддингу (вектору слова) добавляется небольшой, заранее рассчитанный вектор, который кодирует его позицию в последовательности (например, первый, второй, третий токен). Эти позиционные векторы имеют специальную структуру из синусоид и косинусоид, что позволяет модели легко интерполировать позиции для слов, которых не было в обучающей выборке [1]. Таким образом, каждый токен теперь представлен не просто своим смыслом, но и своим местом в предложении.

Теперь последовательность векторов-эмбеддингов с позиционной информацией отправляется в основную часть архитектуры — блок **Encoder** (кодировщик). Архитектура Transformer состоит из двух основных частей: Encoder, который обрабатывает входные данные (запрос), и Decoder (декодировщик), который генерирует выходные данные (ответ) [1] [3]. Encoder представляет собой стопку из нескольких одинаковых слоев (например, 6 в оригинальной модели), которые обрабатывают весь входной текст одновременно, а не по частям [32]. Каждый слой Encoder содержит два ключевых подслоя: механизм **Self-Attention** и **Feed-Forward Neural Network** (FFNN) [1].

FFNN — это простая нейронная сеть, которая применяется независимо к каждому токену в последовательности. Однако главным событием является Self-Attention. Этот механизм позволяет каждому токену анализировать все остальные токены в предложении и решать, на какие из них следует "обратить внимание" для лучшего понимания своего собственного значения. Например, когда модель обрабатывает местоимение "она", Self-Attention поможет ей связать это слово с существительным "кошка", которое встретилось ранее в предложении. Эта возможность рассматривать всю последовательность сразу, а не только предыдущие слова, является ключевым преимуществом трансформеров над предыдущими архитектурами, такими как рекуррентные нейронные сети (RNN) [3]. Выход из одного слоя Encoder становится входом для следующего. Таким образом, информация проходит через несколько уровней абстрагирования, становясь все более отвлеченной и обобщенной.

После того как весь запрос полностью обработан блоком Encoder, его финальное представление (последовательность векторов) передается в блок **Decoder** (декодировщик). Декодировщик также состоит из нескольких (обычно также 6) идентичных слоев, но его работа более сложная и итеративная [1]. Его задача — генерировать ответ слово за словом. Процесс начинается с того, что декодировщик получает на вход специальный токен, обозначающий начало ответа. Затем он применяет свой первый слой Self-Attention, но здесь вступает в действие важное ограничение, называемое маскированием. Чтобы модель не "подсматривала" будущие слова в ответе (которых еще не существует), маска устанавливает нулевой вес для всех токенов, которые еще не были сгенерированы [1] [3]. Это гарантирует, что при генерации первого слова модель может смотреть только на стартовый токен, при генерации второго — на стартовый и первое сгенерированное слово и так далее. После этого подслоя Self-Attention декодировщик использует еще один механизм внимания, который связывает его с выходом Encoder. Это позволяет декодировщику "вспомнить" важные части исходного запроса, когда он генерирует ответ. Например, если запрос был "Переведи 'Hello' на русский", этот механизм поможет декодировщику сфокусироваться на векторе-эмбеддинге слова "Hello", когда он будет выбирать первое слово ответа.

Наконец, после всех сложных манипуляций внутри слоев Decoder, на последнем шаге получается последовательность векторов, представляющих сгенерированный ответ. Каждый вектор в этой последовательности имеет ту же размерность, что и эмбеддинги, но он представляет уже не входное слово, а выходное. Чтобы превратить этот вектор в реальное слово, он проходит через

два последних слоя модели. Сначала через Линейный слой, который просто проецирует высокоразмерный вектор на очень длинный вектор, длина которого равна размеру всей словарной базы модели (может быть десятки тысяч или даже миллионов элементов) [3]. Этот длинный вектор называется вектором "логитов". Затем на него применяется функция **Softmax**. Softmax преобразует этот вектор логитов в вектор вероятностей. Каждый элемент этого вектора показывает вероятность того, что соответствующее ему слово из словаря будет следующим в ответе. Например, если на выходе Softmax получился вектор, где слово "Париж" имеет значение 0.8, а все остальные — меньше 0.1, то модель выбирает "Париж" как следующее слово в ответе. Этот процесс повторяется: сгенерированное слово добавляется к ответу, и декодировщик генерирует следующее слово, пока не сгенерирует специальный токен, обозначающий конец ответа. Таким образом, генерация ответа — это не одно решение, а процесс принятия многочисленных статистически обоснованных решений, где на каждом шаге модель выбирает наиболее вероятное продолжение, основываясь на своем глубоком "понимании" языка, полученном в процессе обучения.

Механизм внимания: Сердце параллельной семантической аналитики

Механизм внимания, и в частности его вариант "само-внимание", является не просто одним из компонентов архитектуры Transformer, а ее истинным "сердцем" и главным фактором, обеспечившим ей успех [30]. Этот механизм кардинально меняет подход к обработке последовательной информации, такой как текст, и позволяет модели одновременно анализировать все элементы входной последовательности, находя между ними сложные зависимости. Чтобы понять его силу, необходимо сначала рассмотреть проблему, которую он решает, а затем разобрать его внутреннюю работу на концептуальном уровне.

Проблема, с которой сталкивались предыдущие популярные архитектуры, такие как рекуррентные нейронные сети (РНС), заключалась в их последовательной природе. РНС обрабатывают текст слово за словом, и информация о предыдущих словах "хранится" в своих внутренних состояниях. Это эффективно для связей между соседними словами, но создает проблемы с длинными зависимостями. Чем дальше находится релевантное слово от текущего, тем

сложнее РНС "вспомнить" его. Информация со временем "исчезает" или искажается в процессе передачи. Кроме того, последовательная обработка делает обучение РНС медленным, так как компьютеры не могут эффективно выполнять эти шаги параллельно. Transformer решил обе эти проблемы, отказавшись от рекурсивной обработки в пользу полностью параллельного подхода [17].

Концептуально, механизм Self-Attention можно представить как систему коллективного анализа, где каждый элемент (слово) в последовательности посылает запросы другим элементам, чтобы понять их контекст. Для этого каждый токен, представленный своим эмбеддингом, генерирует три новые версии своего вектора: вектор запроса, вектор ключа и вектор значения [1] [3]. Эти три вектора создаются путем умножения исходного эмбеддинга на три разные матрицы весов (Q, K, V), которые модель "выучила" во время обучения. Интерпретация этих векторов такова:

- **Query (Запрос):** Представляет собой то, что данный токен "ищет" в других токенах. Например, если токен — это местоимение "оно", его Query-вектор будет содержать информацию о том, что он ищет антецедент, то есть существительное.
- **Key (Ключ):** Представляет собой то, что данный токен "предлагает" другим токенам. Он служит маркером, по которому другие токены могут идентифицировать его как потенциально релевантный.
- **Value (Значение):** Это сама суть токена, его семантическая информация, которая будет использоваться, если токен будет "выбран" другими.

Процесс расчета внимания для одного токена (отправителя) выглядит следующим образом: 1. Счетчики внимания: Для каждого другого токена (получателя) в последовательности вычисляется "оценка внимания". Это делается путем вычисления "сходства" между Query-вектором отправителя и Key-вектором получателя. Чаще всего это делается с помощью скалярного произведения (dot product) [1] [3]. Чем выше результат, тем больше отправитель "интересуется" этим получателем. 2. Нормализация: Все оценки внимания, рассчитанные для одного отправителя ко всем остальным токенам, проходят через функцию Softmax. Это превращает их в набор чисел от 0 до 1, которые в сумме дают единицу. Теперь они можно интерпретировать как вероятности или степени уверенности [1]. Токен, на который обращено наибольшее внимание, получает вероятность, близкую к 1. 3. Взвешенное суммирование: Наконец, для получения нового, "проанализированного" вектора для отправителя, берутся все Value-векторы всех токенов в последовательности. Каждый Value-вектор

умножается на свою соответствующую "вероятность внимания". Затем все полученные векторы суммируются [3]. Токен, на который было обращено большое внимание, вносит больший вклад в итоговый вектор, а токены, игнорируемые моделью, — меньший.

В результате этого процесса каждый токен в последовательности заменяется новым вектором, который является своего рода "синтезом" всего предложения, но с акцентом на информацию, наиболее релевантную именно этому токену. Например, после прохождения Self-Attention, вектор для слова "it" в предложении "The animal didn't cross the street because it was too tired" будет сильно похож на вектор для слова "animal", так как модель научилась установить эту связь [1].

Чтобы еще больше повысить мощь этого механизма, в архитектуре Transformer используется **Multi-Head Attention (**Многоголовочное внимание**)** [1] [3]. Вместо одного набора матриц Q, K, V, модель использует несколько (в оригинальной модели — восемь) таких наборов. Каждый набор (или "голова") учится искать разные типы отношений между словами. Например, одна голова может лучше всего находить грамматические связи (подлежащее-сказуемое), другая — семантические (синонимы), третья — временные зависимости. Каждая голова производит свой собственный набор векторов-вниманий. Затем все эти наборы объединяются (конкатенируются) и проходят через еще одну матрицу весов, чтобы получить окончательный, унифицированный вектор вывода [3]. Такой подход позволяет модели одновременно "смотреть" на контекст с нескольких ракурсов, что значительно повышает ее способность к анализу сложных текстов.

Таким образом, механизм внимания — это не просто выбор одного слова, а комплексный процесс взвешивания всей входной информации для каждого токена. Это позволяет Transformer строить богатые, многоуровневые представления текста, где значение каждого слова определяется его отношениями со всеми остальными словами в предложении или даже в документе. Именно эта способность к параллельному глобальному анализу и стала прорывом, позволившей создавать модели, способные к глубокому пониманию языка [2].

Архитектурные слои: Специализированные модули для обработки информации

Архитектура Transformer спроектирована как многоуровневая система, где каждый слой и каждый подслой выполняют свою специфическую функцию, внося вклад в общее понимание и генерацию текста. Эта модульность позволяет модели эффективно обрабатывать сложные языковые конструкции, проходя через серию последовательных преобразований. Основная структура состоит из двух крупных блоков — **Encoder** (кодировщик) и **Decoder** (декодировщик) — каждый из которых состоит из множества повторяющихся подмодулей [1] [3]. Понимание роли каждого из этих слоев и подслоев является ключом к осознанию того, как модель "мыслит" на своем внутреннем языке векторов и матриц.

Блок **Encoder** предназначен исключительно для анализа и кодирования входной последовательности (например, пользовательского запроса). Он состоит из N идентичных слоев, где N обычно равно 6 в классической архитектуре [32]. Важно отметить, что хотя эти слои выглядят одинаково, они не делят между собой веса; каждый слой является самостоятельным и обучается самостоятельно [1]. Каждый слой Encoder содержит два ключевых подслоя: 1. Подслой **Self-Attention:** Как было подробно рассмотрено ранее, этот подслой анализирует всю входную последовательность одновременно, вычисляя, на какие слова следует обратить внимание при обработке каждого отдельного слова. Это позволяет модели улавливать долгие диапазоны зависимостей и семантические связи между словами, расположенными далеко друг от друга [3]. 2. Подслой **FFNN (Feed-Forward Neural Network):** Это простой, но важный компонент. FFNN представляет собой двухслойную нейронную сеть, которая применяется независимо к каждому токену в последовательности, но после того, как Self-Attention уже закончил свою работу [1]. То есть, если Self-Attention занимается "общением" между токенами, то FFNN занимается "внутренней переработкой" информации внутри каждого токена. Он позволяет модели извлекать более сложные и нелинейные признаки из вектора, полученного от Self-Attention. Хотя FFNN применяется к каждому токену по отдельности, его архитектура (размеры слоев, активационные функции) является общей для всей последовательности.

Чтобы сделать обучение глубокой сети из многих слоев стабильным и эффективным, вокруг каждого из этих двух подслоев помещаются два дополнительных элемента, которые применяются ко всему слою целиком:

- **Residual Connections (**Остаточные соединения**):** После каждого подслоя (Self-Attention и FFNN) исходный вектор-вход добавляется к его выходу. Это создает "короткое замыкание" или "обходную дорогу" для информации, позволяя ей "перескакивать" через слой [3]. Это помогает избежать проблемы "деградации сетей", когда добавление новых слоев перестает улучшать производительность или даже ухудшает ее. Стабилизирующее действие residual connections позволяет строить очень глубокие модели, которые успешно обучаются [3].
- **Layer Normalization (**Нормализация по слою**):** Эта техника стандартизирует выходные данные каждого слоя, делая их среднее значение равным нулю, а стандартное отклонение — единице. Это помогает стабилизировать обучение, предотвращая быстрое возрастание или затухание значений в глубоких сетях, и делает процесс обучения более предсказуемым и быстрым [3].

Блок **Decoder** (декодировщик) выполняет противоположную задачу — он генерирует выходную последовательность (ответ). Он также состоит из N идентичных слоев, но его внутренняя структура несколько сложнее. Каждый слой Decoder содержит три подслоя: 1. Подслой **Self-Attention (**с маской**):** Этот подслой работает по тому же принципу, что и в Encoder, но с одной важной поправкой. При генерации ответа модель не должна "подсматривать" будущие слова, которые еще не были созданы. Поэтому на этапе расчета внимания применяется маска, которая обнуляет веса для всех токенов в будущем, заставляя модель полагаться только на уже сгенерированные слова [1] [3]. 2. Подслой **Encoder-Decoder Attention:** Этот подслой работает аналогично Self-Attention, но с ключевой разницей: его Query-векторы берутся из предыдущего подслоя (Self-Attention декодера), а его Key- и Value-векторы берутся от выхода последнего слоя Encoder [1]. Этот механизм позволяет декодировщику "спрашивать" у кодировщика, какие части исходного запроса являются наиболее релевантными для генерации следующего слова в ответе. Это как если бы автор (декодировщик) время от времени заглядывал в свои заметки (выход Encoder), чтобы не потеряться в теме. 3. Подслой **FFNN:** Этот подслой функционально идентичен подслою в Encoder. Он выполняет ту же самую внутреннюю переработку информации для каждого токена в ответе.

Наконец, на выходе из блока Decoder находится последовательность векторов. Чтобы превратить их в слова, как было описано в предыдущем разделе, эти векторы проходят через **Linear layer** (линейный слой), который преобразует их в очень длинный вектор, размер которого соответствует размеру словаря модели, и затем через **Softmax layer** (слой softmax), который преобразует этот вектор в распределение вероятностей, позволяя выбрать наиболее вероятное следующее слово [3]. Эта последовательность слоев, от токенизации до Softmax, составляет полный путь информации через модель, от входа до выхода.

Компонент	Назначение	Контекст использования
Токенизация	Преобразование текста в последовательность числовых идентификаторов.	На этапе обучения и на этапе генерации ответа.
Эмбеддинги	Преобразование токенов в высокоразмерные векторы, кодирующие семантику.	На этапе обучения и на этапе генерации ответа.
Позиционное кодирование	Добавление информации о порядке токенов в последовательности.	На этапе обучения и на этапе генерации ответа.
Encoder (слои)	Анализ и кодирование входной последовательности (запроса).	Только на этапе генерации ответа.
Decoder (слои)	Генерация выходной последовательности (ответа) на основе запроса.	Только на этапе генерации ответа.
Self-Attention	Расчет взаимосвязей между всеми токенами в последовательности.	В каждом слое Encoder и Decoder.
Encoder-Decoder Attention	Связь между декодером и закодированным запросом.	Только в слоях Decoder.
FFNN	Внутренняя переработка информации для каждого токена.	В каждом слое Encoder и Decoder.
Residual Connection & Layer Norm	Стабилизация обучения глубоких сетей.	Окружают каждый подслой в Encoder и Decoder.
Linear + Softmax	Преобразование финального вектора в вероятностное распределение по словарю.	На финальном этапе генерации каждого слова.

Эта четко структурированная иерархия слоев позволяет Transformer эффективно справляться с задачами, требующими глубокого понимания контекста и генерации сложных последовательностей, что и сделало его основой для большинства современных передовых моделей искусственного интеллекта.

Небанальные аналогии и визуализации: Концептуальное понимание сложных систем

Для достижения цели концептуального понимания, особенно для аудитории со средним техническим уровнем, использование аналогий и визуализаций является не просто дополнением, а ключевым инструментом обучения. Исследования в области педагогики и когнитивной психологии показывают, что метафоры и наглядные образы помогают снижать когнитивную нагрузку, делая сложные идеи более доступными и интуитивно понятными [7] [8] [9]. Вместо того чтобы полагаться на банальные и часто вводящие в заблуждение сравнения с человеческим мозгом ("думает", "помнит"), небанальные аналогии должны передавать функциональную роль каждого компонента, а не его внешнее сходство. Ниже представлены несколько таких аналогий, дополняемых концептуальными схемами.

Аналогия **1:** Лазерный сканер и уникальные **QR-**коды**.** Представьте, что модель Transformer — это сверхточный лазерный сканер, предназначенный для цифровизации физических объектов.

- Токенизация**:** Это не просто разрезание текста на слова. Это процесс, при котором лазерный луч сканирует текстовую страницу, как если бы это была поверхность 3D-объекта. Каждый "пиксель" или микро-участок текста, который сканируется, становится отдельным токеном. Но это не просто текст, а текст со своими координатами на странице.
- Эмбеддинги**:** Для каждого полученного токена (например, для слова "столица") сканер не просто запоминает его буквенное написание. Он генерирует для него уникальный, многомерный **QR-**код. Этот QR-код содержит в себе не только само слово, но и всю его семантическую информацию: его род, число, возможные синонимы ("столица" -> "главный город", "столица" -> "столица государства"), ассоциации и контексты, в которых оно встречается в огромной базе данных (датасете). Таким образом, семантически близкие слова будут иметь QR-коды, которые выглядят похоже под микроскопом (в многомерном пространстве).
- Векторы и матрицы**:** Сам QR-код — это и есть вектор. Математические операции, которые модель выполняет с векторами (например, умножение на матрицу), можно представить как различные фильтры или эффекты, применяемые к QR-коду. Умножение на матрицу W_Q — это как применение "фильтра запроса", который выделяет в QR-коде информацию о том, "что искать". Умножение на W_K — "фильтр ключа", который выделяет

информацию о том, "кем являешься". А умножение на W_V — "фильтр значения", который извлекает саму суть объекта [1] [3].

Аналогия **2:** Библиотека-переводчиков и редакторов**.** Эта аналогия, предложенная пользователем, может быть развита в более сложную и динамичную систему [24] [37].

- Целая модель**:** Это не просто библиотека, а целый центр перевода и редактуры. Входящие документы — это запросы пользователя, а выходящие — качественные переводы и написанные на заказ тексты.
- **Encoder (**Переводчики**):** Это команда переводчиков. Их задача — взять входящий документ на одном языке (запрос) и создать максимально точный и глубоко проанализированный перевод на универсальный язык "векторов". Каждый переводчик работает в своей нише: один специализируется на грамматике, другой — на терминологии, третий — на культурных оттенках. Важно, что они работают параллельно над всем документом. У каждого переводчика есть список "знакомых" (это и есть **Positional Encoding**), чтобы они знали, кто стоит рядом, кто — впереди, а кто — позади, и могли беседовать друг с другом, чтобы уточнить контекст [3]. Их главный инструмент — **Self-Attention**, который позволяет им постоянно получать сигналы от коллег о том, какие слова или фразы вызывают неопределенность и требуют уточнения.
- **Decoder (**Редакторы**):** Это команда редакторов, которым нужно написать новый текст на основе предоставленных переводчиками материалов. Редакторы также работают параллельно, но с важным ограничением: они могут видеть только черновики, которые уже написаны, и материалы, предоставленные переводчиками. Их главный инструмент — **Encoder-Decoder Attention**, который позволяет им периодически заглядывать в черновики переводчиков (контекст от Encoder), чтобы убедиться, что они не отклонились от темы. Еще один важный инструмент у редакторов — маскирование. Это своего рода технологическая защита от самообмана: редактор не должен видеть, что он планирует написать в следующем абзаце, чтобы его работа была логичной и последовательной [1] [3].
- **FFNN (**Локальные фильтры**):** После того как редактор получил информацию от переводчиков, он проходит через свой личный "офис-фильтр" (FFNN), где он очищает, уточняет и усиливает свою идею, прежде чем записать ее в черновик.
- **Softmax (**Выбор лучшего варианта**):** Когда редактор заканчивает черновик, он предлагает несколько вариантов окончания абзаца. Главный редактор (функция Softmax) оценивает их по критериям качества, логики

и стиля, и выбирает самый лучший. Этот процесс повторяется, пока весь текст не будет написан.

Визуализации:

Для наглядного представления этих процессов можно использовать комбинированные визуализации.

1. Схема потока данных (Data Flow Diagram):

- Шаг 1 (Вход): Изображение текстового поля с запросом "Столица Франции?".
- Шаг 2 (Токенизация/QR-код): Стрелка ведет к горизонтальной ленте, разделенной на блоки. Каждый блок помечен токеном ("Столица", "Фран", "##ции") и условным значком QR-кода.
- Шаг 3 (Эмбеддинг + PosEnc): Каждый QR-код блокируется с маленьким флажком с цифрой (позиция 1, 2, 3...).
- Шаг 4 (Прохождение через Encoder): Лента входит в блок "Encoder (Переводчики)". Внутри можно показать, как QR-коды обмениваются сигналами (стрелками) с разными весами, что символизирует механизм Self-Attention. Выход из блока выглядит более "обработанным" и абстрактным.
- Шаг 5 (Прохождение через Decoder): Лента из Encoder входит в блок "Decoder (Редакторы)". Здесь показаны три пути: один путь с подписью "Self-Attention (маска)" ведет назад на уже написанное, путь "Encoder-Decoder Attention" ведет к блокам из Encoder, а FFNN — это прямоугольники-фильтры.
- Шаг 6 (Выход): Лента выходит из Decoder в виде новых QR-кодов. Стрелка ведет к блоку "Линейный + Softmax", который преобразует QR-код в один выбранный токен ("Париж").
- Шаг 7 (Итог): Сгенерированное слово "Париж" добавляется к ответу.

1. Метафорическая диаграмма (Metaphorical Diagram):

- Центральный элемент: Большой, сложный механизм, состоящий из нескольких модулей, обозначенных как "Блок Encoder" и "Блок Decoder".
- Encoder: На изображении "блок Encoder" представлен как комната, полная людей в очках (переводчики). У каждого человека есть планшет (QR-код токена). Они сидят в кругу и постоянно жестикулируют друг другу, смотря на свои планшеты (символизирует Self-Attention). Рядом на стене висит

общий график с цифрами (позиционное кодирование), который все контролируют.
- **Decoder:** "Блок Decoder" — это комната, где люди (редакторы) сидят за письменными столами. У них на столе лежат заметки от переводчиков (связь с Encoder) и они пишут черновики. Между ними протянуты невидимые "канаты" (маска), которые не позволяют им смотреть в будущее. Их работа направлена на создание нового текста на основе старых заметок.
- Связи: Стрелки четко показывают путь информации: от экрана (текст) -> к "токенизатору" -> к "генератору QR-кодов" -> к "блоку Encoder" -> к "блоку Decoder" -> к "выборщику лучших вариантов" -> к экрану (ответ).

Используя эти небанальные аналогии и комбинированные визуализации, можно значительно упростить понимание сложной архитектуры Transformer, сосредоточившись на ее функциональных компонентах и их взаимодействии, вместо того чтобы погружаться в математические детали. Это позволяет достичь поставленной цели — предоставить глубокое, но доступное концептуальное объяснение принципов работы современных языковых моделей.

Справка

1. The Illustrated Transformer（图解Transformer）翻译- 知乎 - 知乎专栏 https://zhuanlan.zhihu.com/p/219714713
2. 图解Transformer - 图灵社区 https://www.ituring.com.cn/article/510704
3. 【笔记】The Illustrated Transformer 学习笔记 - 知乎专栏 https://zhuanlan.zhihu.com/p/697743348
4. 最好的Transformer讲解 - 博客园 https://www.cnblogs.com/abaelhe/p/17477995.html
5. TransforLearn: Interactive Visual Tutorial for the Transformer Model https://www.researchgate.net/publication/375011973_TransforLearn_Interactive_Visual_Tutorial_for_the_Transformer_Model
6. [2211.06441] Metaphors We Learn By - arXiv https://arxiv.org/abs/2211.06441
7. Cognitive Load In Speed-Accuracy Tradeoff - eScholarship https://escholarship.org/uc/item/7bw9q77d

8. (PDF) Challenging Cognitive Load Theory: The Role of Educational ... https://www.researchgate.net/publication/389084470_Challenging_Cognitive_Load_Theory_The_Role_of_Educational_Neuroscience_and_Artificial_Intelligence_in_Redefining_Learning_Efficacy
9. The synergy of pedagogical agents and metaphorical design https://bera-journals.onlinelibrary.wiley.com/doi/10.1111/bjet.70064
10. Characterisation of Cognitive Load Using Machine Learning ... - MDPI https://www.mdpi.com/1424-8220/23/20/8528
11. Supervised Token Attribution from Transformer Attention Patterns https://arxiv.org/html/2601.14112v1
12. Transformer Tokenization Strategies for Network Intrusion Detection https://www.mdpi.com/2073-431X/15/2/75
13. Schematic illustration of the transformer architecture and training... https://www.researchgate.net/figure/Schematic-illustration-of-the-transformer-architecture-and-training-paradigm-The-input_fig2_393903886
14. Transformer-based tokenization for IoT traffic classification across ... https://pmc.ncbi.nlm.nih.gov/articles/PMC12453836/
15. [PDF] Accelerating Training of Transformer-Based Language Models with ... https://www.microsoft.com/en-us/research/wp-content/uploads/2020/11/NeurIPS2020_Progressive_Layer_Dropping.pdf
16. Transformers for Language Translation, Classification and ... https://www.digitalocean.com/community/tutorials/transformers-for-language-translation-classification
17. Overview and challenges of machine translation for contextually ... https://www.sciencedirect.com/science/article/pii/S2589004224021035
18. Deanthropomorphising NLP: Can a language model be conscious? https://pmc.ncbi.nlm.nih.gov/articles/PMC11617003/
19. TransforLearn: Interactive Visual Tutorial for the Transformer Model https://ieeexplore.ieee.org/document/10297598/
20. TransforLearn: Interactive Visual Tutorial for the Transformer Model https://dl.acm.org/doi/10.1109/TVCG.2023.3327353
21. TransforLearn: Interactive Visual Tutorial for the Transformer Model ... https://blog.csdn.net/m0_59402094/article/details/137924096
22. 计算机视觉与模式识别2025_12_23 https://www.arxivdaily.com/thread/75031
23. Proceedings of the 61st Annual Meeting of the Association for ... https://aclanthology.org/volumes/2023.acl-long/
24. Simple Index - Alibaba Cloud https://mirrors.aliyun.com/pypi/simple/

25. [PDF] Machine Learning in Interaction: Tool, Material, Culture - HAL https://hal.science/tel-04170434/file/HDR_manuscript_caramiaux.pdf
26. Audiovisual archives: a practical reader - UNESCO Digital Library https://unesdoc.unesco.org/ark:/48223/pf0000109612
27. The Ultimate Guide to Fine-Tuning LLMs from Basics to Breakthroughs https://arxiv.org/html/2408.13296v1
28. Research on Transformer Fault Diagnosis and Maintenance ... - MDPI https://www.mdpi.com/2227-9717/13/7/1977
29. (PDF) Research on Transformer Fault Diagnosis and Maintenance ... https://www.researchgate.net/publication/392948612_Research_on_Transformer_Fault_Diagnosis_and_Maintenance_Strategy_Generation_Based_on_TransQwen_Model
30. What is an attention mechanism? | IBM https://www.ibm.com/think/topics/attention-mechanism
31. General-Purpose In-Context Learning by Meta-Learning Transformers https://arxiv.org/html/2212.04458v2
32. TMAC: Training-targeted Mapping and Architecture Co-exploration ... https://ieeexplore.ieee.org/iel8/10410247/10572272/10787435.pdf
33. Fine-Tuning 164M-Parameter Transformer on Alpaca GPT-4 Dataset ... https://www.linkedin.com/posts/siddharth-sachar-2816b013_part-8-building-a-gpt-like-transformer-from-activity-7405276090140884992-Ot1X
34. Structured Multidimensional Representation Learning for Large ... https://arxiv.org/html/2603.05727v1
35. [PDF] Proceedings of the 19th Sound and Music Computing Conference https://inria.hal.science/hal-03927188v1/file/proceedings.pdf
36. [PDF] Proceedings of the Third Workshop on Computational Typology and ... https://aclanthology.org/2021.sigtyp-1.pdf
37. AI Innovations in Library Services | PDF | Artificial Intelligence - Scribd https://www.scribd.com/document/843383536/IA-Libraries-10-1515-9783111336435